

WinForms/Avalonia and .NET
Cpt S 321 Homework Assignment
Washington State University

Read the entire HW before starting to work on it.

Overview of the steps to follow:

1. Create a branch called "Branch_HW2" and work in this branch for this assignment.
2. Read the Homework **Grading Rubric and the Points Breakdown** that appear at the end of this document.
3. **Work on the Assignment Instructions below.**
4. **Create a pdf document with your AI statement for the HW (call it "AI_HW2"), stating whether or not you used GenAI and how and upload it in the "AI_Statements" folder. If you used GenAI for this HW, 1) detail the process you followed in this document and 2) follow the guidelines for committing code generated by AI and adapted by you as outlined in the syllabus in the AI Statement section.**
5. Once you are done and before the deadline, tag the version that you would want us to grade with the assignment number. For example, "HW1", "HW2", etc.
6. **Record a video to demo your HW (call it "Demo_HW2-FirstName_LastName").**
7. On Canvas -> Assignments -> Assignments -> **Upload your pre-recorded video by the HW deadline.**
8. Sign up for a live demo with the TAs before the deadline.

Assignment Instructions:

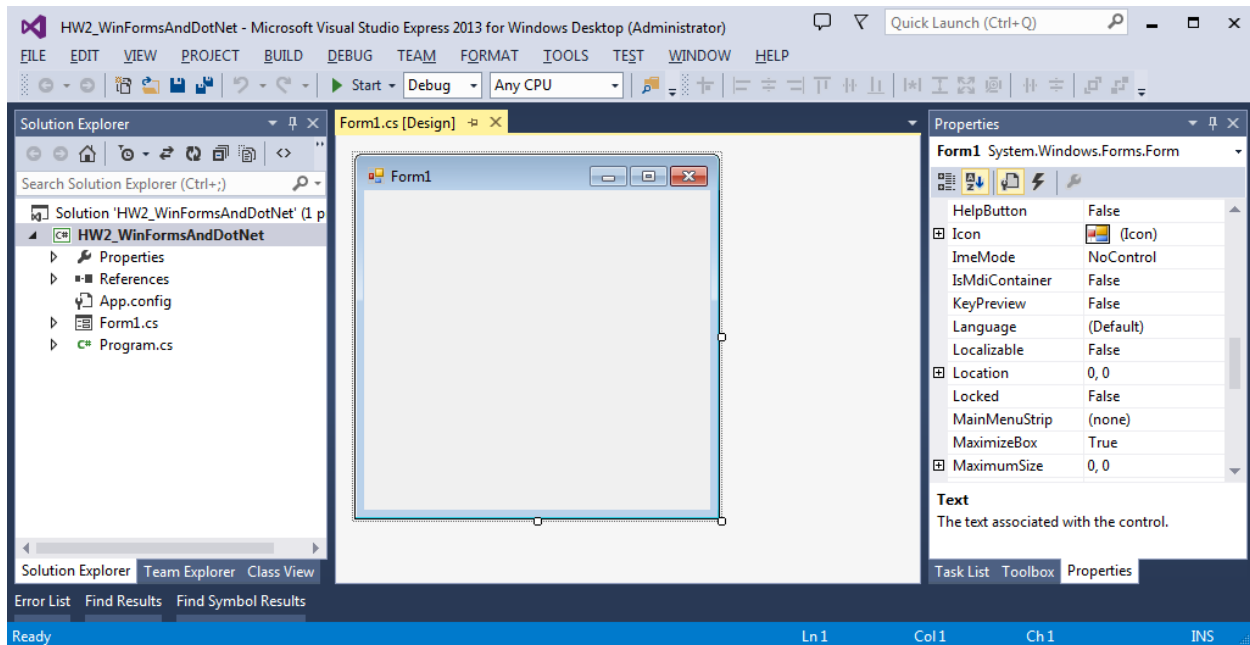
Read all the instructions *carefully* before you write any code.

A. If you are using WINFORMS, follow the instructions below:

Using Visual Studio, create a new WinForms application:

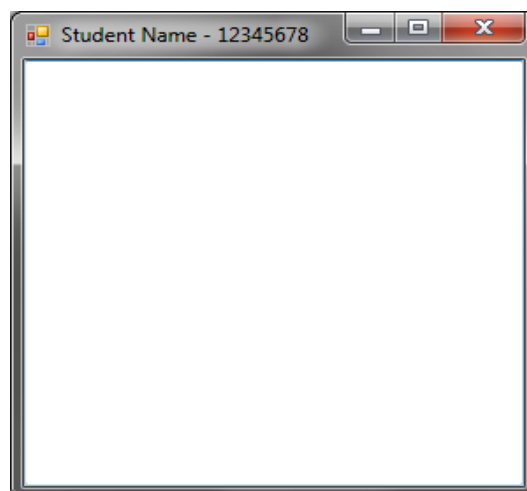
- In VS create a new project (Right click on the Solution -> Add -> New Project)
- Make sure you have Visual C# -> Windows Desktop selected in the left pane.
- Select "Windows Forms App" (for older versions you might only see "Windows Forms App (.NET Framework)" which is what you want)

You will see the WinForms designer appear and it will look something like the following:



In this application you will just be doing computations with simple text output for the results, but instead of displaying such results in the console/terminal window, you'll be displaying them in a text box within your interface. Add a TextBox control from the Toolbox View (View -> Toolbox if you can't see it) into your interface – you do this by dragging and dropping from the toolbox to the form (the middle view in the above screenshot). Using the Property View (the right view in the above screenshot), set the [Multiline](#) property to true and the [Dock](#) property to "Fill"; if you do not see the multiline property you might not have the textbox selected so make sure you select it first. Also set the [Text](#) property of your form to your name and ID #, so that it appears in the title bar; again here make sure you select the form and then look in the Properties View.

After constructing the interface but before writing code, you can run the app to see what it looks like. Your interface should look something like the following at this point:



Next, create a method in the form that you would want to be executed after the form loads. This is where you will be calling the methods for the processing tasks described below. Note that we say “calling the methods” which means that we expect you to organize your code into classes and methods using object-oriented design principles where methods and classes are cohesive elements.

Visual Studio will flip over to your code at this point and your form class will look something like the following:

```
public partial class Form1 : Form
{
    public Form1()
    {
        InitializeComponent();
        RunDistinctIntegers();
    }

    private static void RunDistinctIntegers() // this is your method
    {
    }
}
```

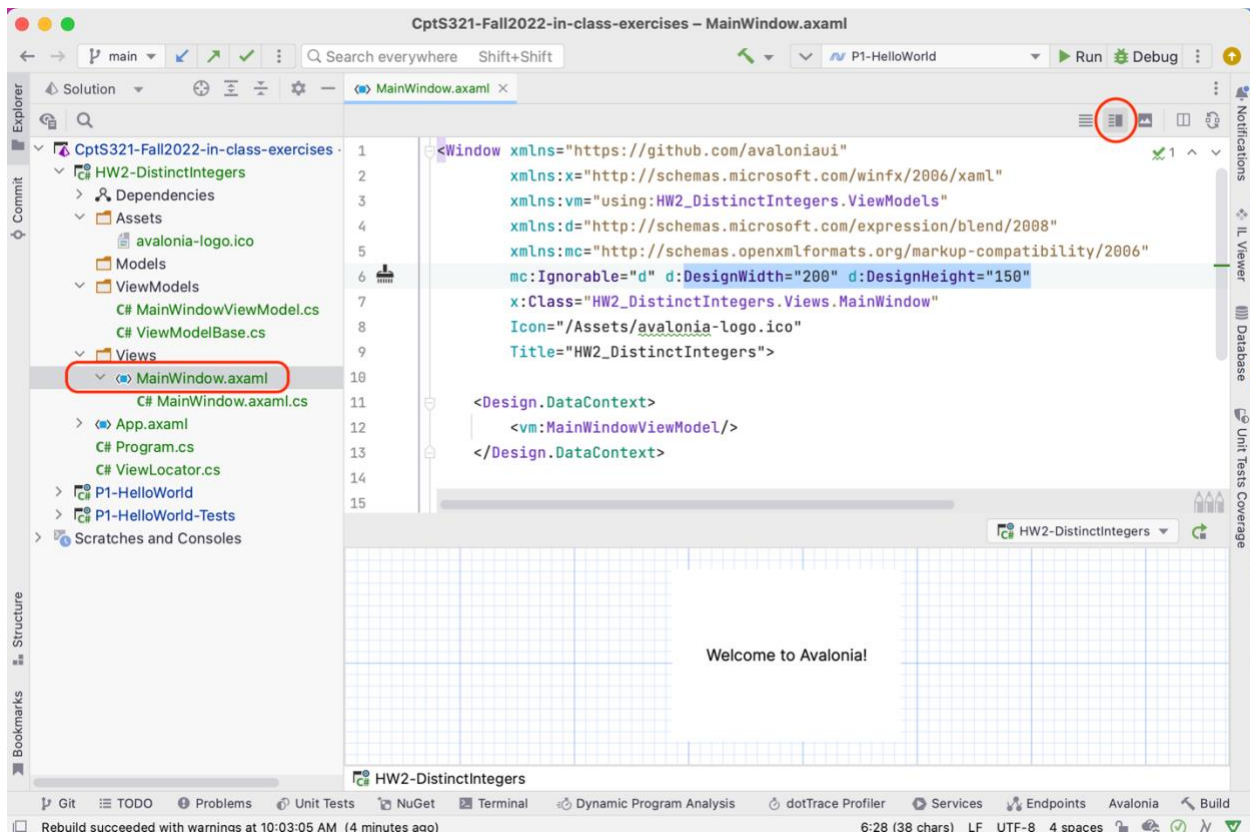
B. If you are using AVALONIA, follow the instructions below:

Install and setup Avalonia in JetBrains Rider: <https://docs.avaloniaui.net/docs/getting-started/ide-support/jetbrains-rider-setup>

Using Rider, create a new Avalonia application:

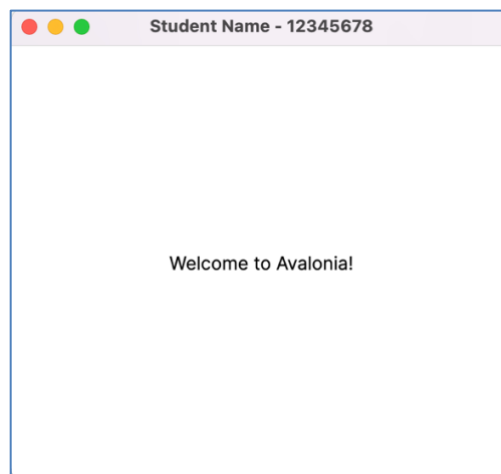
- In Rider create a new project (Right click on the Solution -> Add -> New Project)
- Select “Avalonia .NET Core MVVM App”

You will see the new application appear and it will look something like the following:



In this application you will just be doing computations with simple text output for the results, but instead of displaying such results in the console/terminal window, you'll be displaying them in a text box within your interface. By default Avalonia has created a TextBlock into your interface – where the text “Welcome to Avalonia” is displayed. In the MainWindow.axaml file, set the text of the title property of the main window to your name and ID #, so that it appears in the title bar.

After constructing the interface but before writing code, you can run the app to see what it looks like. Your interface should look something like the following at this point:



You can remove the horizontal and vertical center alignment for the text block and place a small padding:

```
<TextBlock Text="{Binding Greeting}" Padding="10"/>
```

Next, create a method in the form that you would want to be executed after the form loads. This is where you will be calling the methods for the processing tasks described below. Note that we say “calling the methods” which means that we expect you to organize your code into classes and methods using object-oriented design principles where methods and classes are cohesive elements.

If you open your MainWindowViewModel.cs, adapt your code to look something like the following:

```
public class MainWindowViewModel : ViewModelBase
{
    public MainWindowViewModel()
    {
        Greeting = RunDistinctIntegers();
    }

    private string RunDistinctIntegers() // this is your method
    {
    }

    public string Greeting { get; set; }
}
```

Once you have completed A or B above, follow the instructions below.

Use the [Random](#) class to create a list (System.Collections.Generic.List) with 10,000 random integers in the range [0, 20,000] (give or take a few hundred in that range is fine). Then determine how many distinct integers are in the list with 3 different approaches. Also, have them run in the order listed below. Do not change the order.

To be clear: You are not allowed to use ready implementations of Distinct. A note on what you’re doing: you’re taking an array of numbers and conceptually just removing duplicates and counting how many are left. If the input array was {1,1,3,5,6,6,7,7,9} then the distinct number set is {1,3,5,6,7,9}, implying 6 distinct numbers.

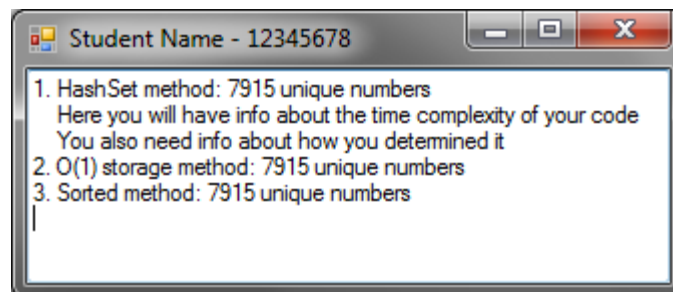
1. Do not alter the list in any way and use a hash set to determine the number of distinct integers in the list. The result will be included in the output, which is discussed more below. Also, include in the output information about the time complexity of this method. Be careful with this and describe how you determined it. Use a good amount of detail, as a too sparse explanation might not be worth full points. Use the MSDN documentation to assist you.
2. Do not alter the list in any way and determine the number of distinct items it contains while keeping the storage complexity (auxiliary) at $O(1)$. This means that you cannot dynamically allocate any memory. If you have an algorithm that would require more storage if the list had

more items, then it is not $O(1)$ storage complexity. Moreover, you cannot allocate additional lists, arrays, or containers of any sort. There is some leniency on the time complexity requirements for this part. Due to the strict memory requirements your method may end up being quite slow next to the other 2, but it still should execute in a matter of a few seconds on any computer made within the last 5 years.

- Sort the list (use built-in sorting functionality) and then use a new algorithm to determine the number of distinct items with $O(1)$ storage, no dynamic memory allocation, and $O(n)$ time complexity. Do not alter the list further after sorting it. Determine the number of distinct items in $O(n)$ time (not including the sorting time, which you can ignore), where n is the number of items in the list.

Output:

Use either a string or StringBuilder object to compile text results from all of your methods. Put this string in the text box after completion, so that it shows up when the app runs. Examine your results before submitting your code and make sure they make sense. Obviously, all 3 methods should be giving you the same answer. It should look something like the following (but of course with more details filled in where I have neglected to give you the answer) when you're finished:



The ~~strikethrough~~ text below means that this is something that we will be adding in later assignments but you are free to skim through and work on them if you feel up for the task.

Homework Grading Rubric and Points Breakdown	
CR1: Functionality (4 points)	- Fulfill all the requirements above with no inaccuracies in the output and no crashes. 1 points for correct implementation of part 1 1.5 points for correct implementation of part 2 1.5 points for correct implementation of part 3
CR2 Quality of Version Control (1 point)	- Homework should be built iteratively (i.e., one feature at a time, not in one huge commit). - Each commit should have cohesive functionality. - Commit messages should concisely describe what is being committed. - TDD should be used (i.e, write and commit tests first and then implement and commit functionality).

	● Include “TDD” in all commit messages with tests that are written before the functionality is implemented. ● Use of a .gitignore. ● Commenting is done as the homework is built (i.e, there is commenting added in each commit, not done all at once at the end).
CR3: Quality of Identifiers (1 point)	● No underscores in names of classes, attributes, and properties. ● No numbers in names of classes or tests. ● Identifiers should be descriptive. ● Project names should make sense. ● Class names and method names use PascalCasing. ● Method arguments and local variables use camelCasing. ● No Linguistic Antipatterns or Lexicon Bad Smells.
CR4: Existence and Quality of Comments (1 point)	● Every method, attribute, type, and test case has a leading comment block with a minimum of <summary>, <returns>, <param>, and <exception> filled in as applicable. ● All comment blocks use the format that is generated when typing “///” on the line above each entity. ● There is useful inline commenting <u>in addition to leading comment blocks</u> that explains how the algorithm is implemented as needed.
CR5: Quality of Code (1 point)	● Each file should only contain one public class. ● Correct use of access modifiers. ● Classes are cohesive. ● Namespaces make sense. ● Code is easy to follow. ● StyleCop is installed and configured correctly for all projects in the solution and all warnings are resolved. If any warnings are suppressed, a good reason must be provided. ● Use of appropriate design patterns and software principles seen in class.
CR6: Existence and Quality of Tests (1 point)	● Normal, boundary, and overflow/error cases should be tested – tests should be written with NUnit. ● Test should be modularized (i.e, have a separate test case for each functionality - do not combine them into one large test case). ● <i>Note: In assignments with a GUI, we do not require testing of the GUI itself.</i>
CR7: Demo video – 5 min max (1 point)	● Check the “Pre-recorded demos instructions” in the additional resources for details on the items below.

	• Version control history overview (< 30 sec.) • AI statement (< 30 sec.) • Design of the solution (2 min max) • Features showcase (2 min max)
--	--